CAPSTONE PROJECT REPORT

AI-Based Hand Gesture Recognition System

Computer Vision • Machine Learning • Real-Time Prediction

Project Title	AI-Based Hand Gesture Recognition System
Primary Language	Python
Core Libraries	OpenCV, MediaPipe, scikit-learn
Interface	Webcam-based real-time demo
Prepared for	Capstone project submission
Student / Department / Institution	Fill in your details before submission

Implementation note: the report includes runnable starter code and clearly labelled illustrative output. Actual accuracy, confusion matrix, and screenshots must be generated by running the project with your own collected dataset.

Contents

1. Abstract and problem statement

2. Objectives and scope

3. System requirements and tools

4. Proposed architecture

5. Dataset collection, preprocessing and EDA

6. Model training and evaluation

7. Real-time implementation

8. Output and demonstration evidence

9. Testing, limitations and future enhancement

10. Conclusion and references

Appendix A. Complete source code

Appendix B. Run instructions and submission checklist

Important: this is a reproducible project template. Do not present the example metrics or mock interface as measured results. Replace them with results produced on your own system.

1. Abstract

This project develops a real-time hand gesture recognition system using a webcam, OpenCV, MediaPipe hand landmarks, and a supervised machine-learning classifier. The camera captures frames; MediaPipe locates a hand and returns 21 landmark points. The landmarks are converted into a normalized feature vector, and a trained Random Forest classifier predicts a predefined gesture label. The result is displayed on the video frame. A small labelled dataset can be collected using the included data-collection script, after which the training script evaluates the model on a held-out test set and saves the trained model.

Problem statement

Build an AI system that recognizes predefined hand gestures from camera images or video, maps each gesture to a label, evaluates classification performance on unseen samples, and demonstrates predictions in real time.

Why this project matters

Hand gestures can provide a touch-free input method for human-computer interaction. The same general approach can support accessibility prototypes, classroom demonstrations, smart interfaces, and simple device-control prototypes. This project is an educational prototype and should not be treated as a safety-critical controller.

Expected deliverables

• Source code for collecting labelled data, training a classifier, and running live recognition.

• Dataset folder with gesture labels and feature vectors.

• EDA summary and model evaluation metrics: accuracy, precision, recall, F1-score and confusion matrix.

• Real-time webcam demonstration with the predicted label displayed.

• Output screenshots/video, this project report and a presentation.

2. Objectives and scope

• Detect one hand in webcam frames and extract 21 landmarks using MediaPipe.

• Create a labelled dataset for four example gestures: FIST, OPEN_PALM, PEACE and THUMBS_UP.

• Normalize landmark coordinates to reduce the effect of hand position and image scale.

• Train a Random Forest classifier and evaluate it on a stratified test split.

• Display the predicted gesture and confidence-like class probability in a live webcam window.

• Document practical limitations, ethical considerations and future improvements.

Scope

The starter implementation classifies one visible hand and four gestures. It does not interpret arbitrary sign language, infer intent, or guarantee reliable performance in all lighting, backgrounds, camera angles, skin tones, or hand sizes. Gestures must be collected and labelled consistently.

3. System requirements and tools

Component	Recommended setup / purpose
Operating system	Windows, Linux, or macOS
Python	Python 3.10 or 3.11 recommended for package compatibility
OpenCV	Webcam capture, frame conversion and drawing overlays
MediaPipe	Hand detection and 21-point landmark extraction

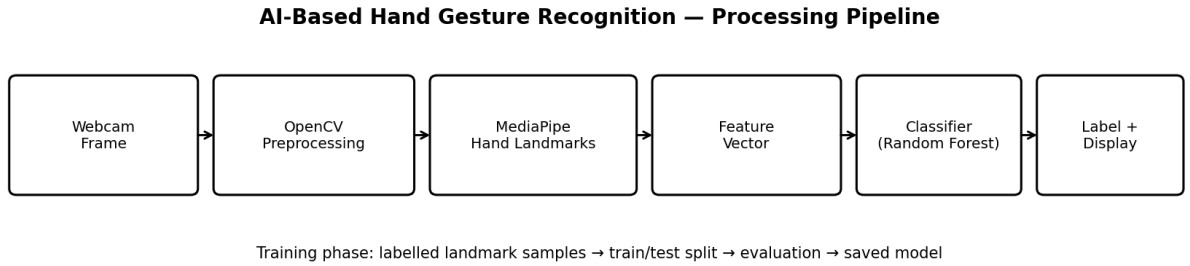
Component	Recommended setup / purpose
scikit-learn	Random Forest, train/test split and evaluation metrics
NumPy / pandas	Feature arrays and dataset management
Matplotlib	EDA charts and confusion-matrix visualization
Hardware	Laptop/desktop with webcam; CPU is sufficient for this small demo

Install dependencies

```
python -m venv .venv
# Activate the environment, then:
python -m pip install --upgrade pip
pip install opencv-python mediapipe scikit-learn numpy pandas matplotlib joblib
```

If MediaPipe does not install for your Python version, create an environment with Python 3.10 or 3.11 and retry. Avoid installing packages globally if you can use a virtual environment.

4. Proposed architecture



Processing stages: (1) capture frame, (2) convert BGR to RGB, (3) detect hand landmarks, (4) normalize and flatten the landmark coordinates, (5) pass the feature vector to the saved classifier, and (6) display the predicted class on the frame.

Landmark features

MediaPipe returns 21 landmarks per detected hand. Each landmark has x, y, and z values, giving 63 values in total. The implementation subtracts the wrist landmark coordinates from every point and scales the x/y coordinates by the largest absolute x/y offset. This reduces sensitivity to translation and hand size. The z values are shifted relative to the wrist. This simple normalization is useful for a prototype but does not remove all rotation or viewpoint variation.

5. Dataset collection, preprocessing and EDA

Gesture classes

Class label	Example gesture	Collection tip
FIST	Closed fist	Keep fingers curled and thumb position consistent
OPEN_PALM	Open hand, fingers extended	Keep palm facing the camera
PEACE	Index and middle fingers extended	Keep other fingers folded
THUMBS_UP	Thumb extended upward	Keep other fingers folded

Collection procedure

• Create a folder named data and run collect_data.py.

• Choose one class when prompted, show that gesture to the webcam, and press C to save a sample.

• Collect a few hundred samples per class if practical, with small changes in distance, position, lighting and background.

• Press Q to stop. Repeat for each class. Do not collect only near-identical frames; variation improves generalization.

• Use a balanced number of samples per class. The training script reports the observed class counts.

Preprocessing and exploratory data analysis

Only frames in which MediaPipe detects a hand are saved. Each sample is a 63-value normalized landmark vector with a class label. The training script reports the total sample count, per-class counts, missing values, train/test split sizes and classification report. EDA should be used to identify class imbalance and collection errors before relying on the model.

Privacy

The default dataset stores numerical landmarks rather than full camera images. Still, collect data only with permission, avoid recording identifiable bystanders, and keep the local dataset secure. Delete samples that were collected accidentally or without consent.

6. Model training and evaluation

Model choice

A Random Forest classifier is a practical baseline for a small structured dataset. It learns a set of decision trees from the landmark feature vectors and combines their predictions. It trains quickly on a laptop and requires less data and compute than training a CNN from raw images.

Evaluation protocol

• Use a stratified 80/20 train/test split so each gesture is represented in both partitions.

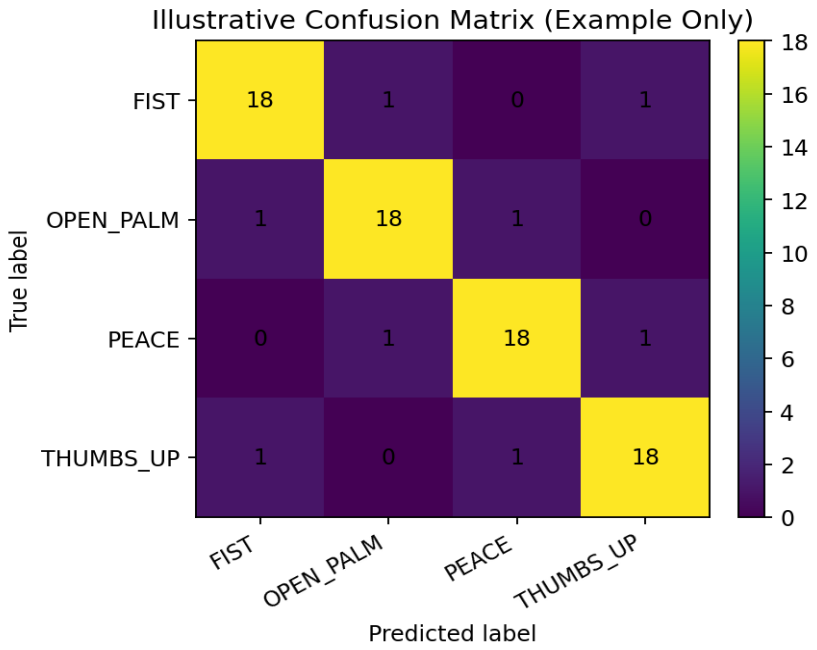
• Train only on the training partition; use the test partition for a final estimate on unseen samples.

• Report accuracy, precision, recall and F1-score. For multiclass classification, include macro averages.

• Inspect the confusion matrix to see which gestures are confused with each other.

• If multiple frames from one recording session are nearly identical, random splitting may overestimate performance. For stronger evaluation, record separate sessions and hold out an entire session.

Metric definitions: Accuracy = correct predictions / all predictions. Precision for a class = true positives / all predicted positives for that class. Recall = true positives / all actual positives for that class. F1-score is the harmonic mean of precision and recall. Macro averages give each class equal weight.



The matrix above is illustrative only. Replace it with the matrix generated by train_model.py. It must not be reported as an actual experimental result.

7. Real-time implementation

The live demo loads gesture_model.joblib and label_encoder.joblib, opens the default webcam, detects landmarks in each frame, applies the same normalization used in training, and displays the predicted class. Press Q to quit. If no hand is detected, the display shows 'No hand detected'.

Live demo steps

• Collect data for all classes.

• Train and evaluate the model.

• Run python realtime_demo.py.

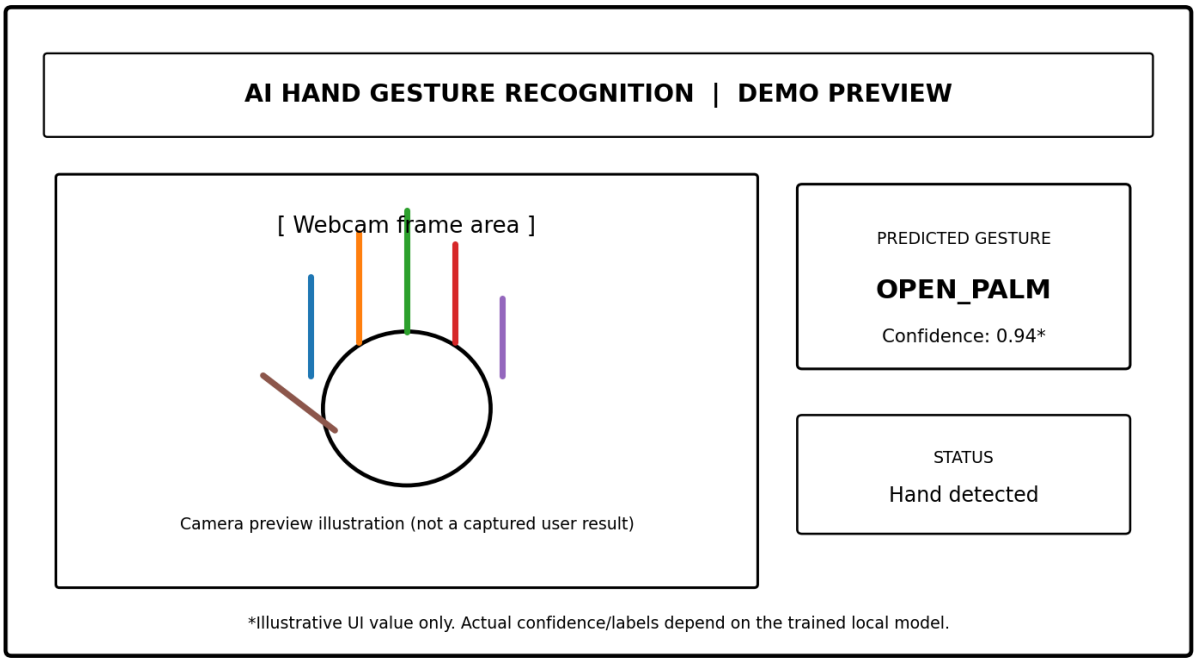
• Show one of the trained gestures clearly to the camera.

• Capture screenshots of several correctly and incorrectly classified examples for the report.

• Record the final measured metrics and describe conditions under which the demo works or fails.

8. Output and demonstration evidence

The following mock screen illustrates the intended layout. It is not a screenshot from an executed model, and its label/confidence are placeholders.



Expected console output format

```
Dataset loaded: 800 samples, 4 classes
Train samples: 640 | Test samples: 160
Class counts:
FIST        200
OPEN_PALM   200
PEACE       200
THUMBS_UP   200

Accuracy: <measured value from your run>
Classification report:
              precision  recall  f1-score  support
FIST             ...      ...      ...       ...
OPEN_PALM        ...      ...      ...       ...
PEACE            ...      ...      ...       ...
THUMBS_UP         ...      ...      ...       ...
```

The sample counts above are an example of a balanced dataset, not a claim about your collected data. The included code prints the actual counts and metrics. Copy those actual values into the final report after running the project.

Screenshots to include in your final submission

• Screenshot 1: data collection window showing a gesture and saved-sample counter.

• Screenshot 2: terminal output showing dataset counts and evaluation metrics.

• Screenshot 3: actual confusion matrix produced by the training script.

• Screenshot 4: real-time demo correctly predicting at least two different gestures.

• Screenshot 5: a difficult condition or misclassification, followed by a short explanation.

9. Testing, limitations and future enhancement

Test cases

Test	Input / condition	Expected behavior
T1	Hand clearly visible, OPEN_PALM	Predict OPEN_PALM if represented well in training data
T2	Hand clearly visible, FIST	Predict FIST if represented well in training data
T3	No hand in frame	Display 'No hand detected'
T4	Dim light / partial hand	May be uncertain; record outcome

Test	Input / condition	Expected behavior
T5	Gesture not in trained classes	May incorrectly map to a known class; system has no unknown-class detector
T6	Webcam unavailable	Show an understandable error and exit

Limitations

• Performance depends on the quality, diversity and balance of the collected data.

• Occlusion, motion blur, unusual hand orientation, low light and cluttered backgrounds can reduce detection accuracy.

• The classifier always chooses among trained classes when a hand is detected; it may confidently mislabel unfamiliar gestures.

• Landmark-based recognition can be sensitive to camera viewpoint and landmark jitter.

• Results measured on one person's hand may not generalize to all users.

Future enhancements

• Add more gestures and more users, backgrounds, lighting conditions and camera angles.

• Use session-based validation and cross-user testing.

• Add temporal smoothing across multiple frames to reduce flickering labels.

• Add an unknown-gesture rejection threshold and a confidence display calibrated on validation data.

• Create an optional Streamlit dashboard for dataset statistics and predictions.

• Map gestures to safe, low-voltage demonstration actions only, with manual override and clear safeguards.

10. Conclusion

This project demonstrates a complete introductory computer-vision workflow: landmark extraction, labelled data collection, feature preprocessing, supervised learning, quantitative evaluation and real-time prediction. The Random Forest model provides a lightweight baseline for recognizing a small set of hand gestures. The quality of the final result must be established by running the supplied code on a varied local dataset and reporting the measured evaluation metrics, not by using the illustrative values or mock output in this document.

References

• MediaPipe documentation — Hand landmarks and hand tracking: https://developers.google.com/mediapipe

• OpenCV documentation — Video capture and image processing: https://docs.opencv.org/

• scikit-learn documentation — RandomForestClassifier and model evaluation: https://scikit-learn.org/stable/

• Python documentation: https://docs.python.org/3/

Appendix A. Complete source code

Create the following three Python files in one project folder. The code below is a compact runnable baseline. The same normalization function is used during data collection, training and real-time inference.

File 1 — common.py

```
import numpy as np

def normalize_landmarks(hand_landmarks):
    """Return 63 normalized x/y/z landmark features, or None."""
    if hand_landmarks is None:
        return None
    pts = np.array([[lm.x, lm.y, lm.z] for lm in hand_landmarks.landmark],
                   dtype=np.float32)
    if pts.shape != (21, 3):
        return None
    # Translation normalization relative to wrist landmark (index 0).
    pts = pts - pts[0]
    # Scale x/y by the largest x/y offset; protect against division by zero.
    scale = float(np.max(np.abs(pts[:, :2])))
    if scale > 1e-6:
        pts[:, :2] /= scale
    # z is already wrist-relative; use a robust scale for similar magnitude.
    zscale = float(np.max(np.abs(pts[:, 2])))
    if zscale > 1e-6:
        pts[:, 2] /= zscale
    return pts.flatten().astype(np.float32)
```

File 2 — collect_data.py

```
import os
import csv
import cv2
import mediapipe as mp
from common import normalize_landmarks

DATA_DIR = "data"
CSV_PATH = os.path.join(DATA_DIR, "landmarks.csv")
CLASSES = ["FIST", "OPEN_PALM", "PEACE", "THUMBS_UP"]

os.makedirs(DATA_DIR, exist_ok=True)
new_file = not os.path.exists(CSV_PATH)
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise RuntimeError("Could not open webcam. Check camera permissions.")

mp_hands = mp.solutions.hands
mp_draw = mp.solutions.drawing_utils
saved = 0

print("Classes:", ", ".join(CLASSES))
label = input("Type the class to collect exactly as shown above: ").strip().upper()
if label not in CLASSES:
    cap.release()
    raise ValueError("Unknown class label.")

with open(CSV_PATH, "a", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    if new_file:
        writer.writerow(["label"] + [f"f{i}" for i in range(63)])

    with mp_hands.Hands(
        static_image_mode=False,
        max_num_hands=1,
        min_detection_confidence=0.6,
        min_tracking_confidence=0.5
    ) as hands:
        while True:
            ok, frame = cap.read()
            if not ok:
                print("Camera frame could not be read.")
                break
            frame = cv2.flip(frame, 1)
            rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            result = hands.process(rgb)

            feature = None
            if result.multi_hand_landmarks:
                hand = result.multi_hand_landmarks[0]
                mp_draw.draw_landmarks(frame, hand, mp_hands.HAND_CONNECTIONS)
                feature = normalize_landmarks(hand)

            cv2.putText(frame, f"Class: {label} | Saved: {saved}",
                        (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7,
                        (0, 255, 0), 2)
```

```
            cv2.putText(frame, "Press C to capture | Q to quit",
                        (10, 60), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                        (255, 255, 255), 2)
            cv2.imshow("Collect Gesture Data", frame)
            key = cv2.waitKey(1) & 0xFF

            if key == ord("c") and feature is not None:
                writer.writerow([label] + feature.tolist())
                f.flush()
                saved += 1
                print(f"Saved sample {saved} for {label}")
            elif key == ord("q"):
                break

cap.release()
cv2.destroyAllWindows()
print(f"Finished. Added {saved} samples to {CSV_PATH}")
```

File 3 — train_model.py

```
import os
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (accuracy_score, classification_report,
                             ConfusionMatrixDisplay, confusion_matrix)

CSV_PATH = "data/landmarks.csv"
MODEL_PATH = "gesture_model.joblib"
ENCODER_PATH = "label_encoder.joblib"
os.makedirs("outputs", exist_ok=True)

if not os.path.exists(CSV_PATH):
    raise FileNotFoundError("Run collect_data.py for every gesture first.")

df = pd.read_csv(CSV_PATH).dropna()
if "label" not in df.columns or len(df) < 20:
    raise ValueError("Dataset is too small or missing the label column.")

X = df.drop(columns=["label"]).to_numpy(dtype=np.float32)
y_text = df["label"].astype(str).to_numpy()
if X.shape[1] != 63:
    raise ValueError(f"Expected 63 features, found {X.shape[1]}")

print(f"Dataset loaded: {len(df)} samples, {len(np.unique(y_text))} classes")
print("Class counts:")
print(df["label"].value_counts().sort_index())
print("Missing values:", int(df.isna().sum().sum()))

encoder = LabelEncoder()
y = encoder.fit_transform(y_text)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print(f"Train samples: {len(X_train)} | Test samples: {len(X_test)}")

model = RandomForestClassifier(
    n_estimators=250, random_state=42, class_weight="balanced",
    min_samples_leaf=1, n_jobs=-1
)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, pred):.4f}")
print(classification_report(
    y_test, pred, labels=np.arange(len(encoder.classes_)),
    target_names=encoder.classes_, zero_division=0
))

cm = confusion_matrix(y_test, pred, labels=np.arange(len(encoder.classes_)))
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=encoder.classes_)
disp.plot(cmap="Blues", xticks_rotation=35, values_format="d")
plt.title("Hand Gesture Confusion Matrix")
plt.tight_layout()
plt.savefig("outputs/confusion_matrix.png", dpi=160)
plt.close()

df["label"].value_counts().sort_index().plot(kind="bar")
plt.title("Samples per Gesture Class")
plt.xlabel("Gesture")
plt.ylabel("Number of samples")
plt.tight_layout()
plt.savefig("outputs/class_distribution.png", dpi=160)
plt.close()
```

```
joblib.dump(model, MODEL_PATH)
joblib.dump(encoder, ENCODER_PATH)
print(f"Saved model: {MODEL_PATH}")
print(f"Saved encoder: {ENCODER_PATH}")
print("Saved plots in outputs/")
```

File 4 — realtime_demo.py

```
import os
import cv2
import joblib
import mediapipe as mp
from common import normalize_landmarks

MODEL_PATH = "gesture_model.joblib"
ENCODER_PATH = "label_encoder.joblib"
if not (os.path.exists(MODEL_PATH) and os.path.exists(ENCODER_PATH)):
    raise FileNotFoundError("Train the model first by running train_model.py.")

model = joblib.load(MODEL_PATH)
encoder = joblib.load(ENCODER_PATH)
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise RuntimeError("Could not open webcam. Check camera permissions.")

mp_hands = mp.solutions.hands
mp_draw = mp.solutions.drawing_utils

with mp_hands.Hands(
    static_image_mode=False, max_num_hands=1,
    min_detection_confidence=0.6, min_tracking_confidence=0.5
) as hands:
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        frame = cv2.flip(frame, 1)
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        result = hands.process(rgb)
        text = "No hand detected"

        if result.multi_hand_landmarks:
            hand = result.multi_hand_landmarks[0]
            mp_draw.draw_landmarks(frame, hand, mp_hands.HAND_CONNECTIONS)
            feature = normalize_landmarks(hand)
            if feature is not None:
                pred_id = model.predict([feature])[0]
                label = encoder.inverse_transform([pred_id])[0]
                probabilities = model.predict_proba([feature])[0]
                confidence = float(max(probabilities))
                text = f"{label} ({confidence:.2f})"

        cv2.rectangle(frame, (0, 0), (frame.shape[1], 55),
                      (30, 30, 30), -1)
        cv2.putText(frame, f"Gesture: {text}", (12, 36),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.85, (0, 255, 0), 2)
        cv2.putText(frame, "Press Q to quit", (12, frame.shape[0]-15),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255, 255), 1)
        cv2.imshow("AI Hand Gesture Recognition", frame)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break

cap.release()
cv2.destroyAllWindows()
```

Appendix B. Run instructions and submission checklist

Recommended folder structure

```
hand-gesture-project/
■■■ common.py
■■■ collect_data.py
■■■ train_model.py
■■■ realtime_demo.py
■■■ data/
■   ■■■ landmarks.csv          # created by collection script
■■■ outputs/
■   ■■■ class_distribution.png # created by training script
■   ■■■ confusion_matrix.png   # created by training script
■■■ gesture_model.joblib       # created by training script
■■■ label_encoder.joblib       # created by training script
```

Execution order

```
1. Install dependencies.
2. python collect_data.py
   - Collect one class, quit, then rerun for each remaining class.
   - Use the exact class labels: FIST, OPEN_PALM, PEACE, THUMBS_UP.
3. python train_model.py
4. Inspect outputs/class_distribution.png and outputs/confusion_matrix.png.
5. python realtime_demo.py
6. Take actual screenshots and update the report with measured metrics.
```

Final submission checklist

• Replace student/institution placeholders on the cover page.

• Collect and keep a balanced labelled dataset; check for incorrect labels.

• Run training and save the real classification report and confusion matrix.

• Take genuine screenshots from your own program and insert them into the report/PPT.

• Report actual metrics with test-set size and conditions; do not copy illustrative values.

• Include source files, dataset details, model file, output images, PDF report and final presentation.

• Mention any bugs, limitations and changes you made to the starter implementation.

Troubleshooting: if the camera window is black, close other applications using the camera and check OS permissions. If the model predicts one class repeatedly, collect more varied examples, verify labels, and check class counts. If gestures flicker, collect more samples or implement smoothing over multiple consecutive frames.